Advanced Algorithms Assignment 2:

Comparison Between Randomized (Skip List) and Deterministic (Red-Black Tree) Algorithms

Aaron Falco - 25126703
Laith Alneemy - 24958454

1. The problem and the algorithms

1.1 Overview

Sorted key dictionaries, such as std::map, require a data structure that allows efficient insertion, erasure, and searching, whilst keeping keys sorted and values accessible by reference. Storing key value pairs in sorted vectors causes O(n) insertion and erasure, but allows for O(log n) search through binary search. Storing pairs in singly linked lists allows for O(1) insertion and erasure once the position is found, but O(n) search time, as binary search is impossible on a singly linked list as it cannot be randomly indexed. A binary search tree combines the two: nodes are linked, so insertion and erasure need no shifting, and ordering allows a binary search, so every operation takes time proportional to the tree's height. However, this is only efficient in a balanced binary search tree. Some insertion orders, such as sorted keys, leave the tree unbalanced, and its height and search time grow to O(n). Red-black trees deterministically keep the tree approximately balanced, with no path more than twice as long as another, whilst skip lists achieve the same O(log n) behaviour with layered linked lists, using randomness to keep them balanced in expectation.

1.2 Red-Black Tree

A red-black tree is a binary search tree that stays balanced by storing one extra bit per node, a colour of red or black, and enforcing five properties (Cormen et al., 2009):

1. every node is red or black
2. the root is black
3. every leaf (NIL) is black
4. a red node cannot have a red child
5. every path from a node to its descendant leaves contains the same number of black nodes

Together, the final two properties ensure that the longest root-to-leaf path is no more than twice the length of the shortest, so the height stays below $2 \log_2(n + 1)$ and search, insertion and deletion are O(log n) in the worst case (Cormen et al., 2009). In our implementation every NIL leaf is one shared black sentinel node rather than a null pointer, because the deletion fix-up has to read the colour of, and set the parent of, nodes that may be leaves, and the sentinel lets this happen without special-case checks. Searching ignores colour and

walks down from the root. As our comparator only provides a less-than test, a right step costs two comparisons and a left step one, which is why sorted insertions made roughly twice as many comparisons as reverse-sorted ones in our early benchmarks. A new key is inserted as a red leaf, which preserves the black-height property but can violate the rule against consecutive red nodes. If the new node's uncle is red, recolouring moves the problem two levels up; if the uncle is black, rotations are used to restore the required properties.

When deleting a node with two children, its in-order successor is used to replace it. If the removed node was black, paths through its old position are one black node short, and the fix-up handles this through four cases based on the colour of the sibling and its children, recolouring upwards until the deficit can be absorbed. Both updates therefore run in O(log n) time, with at most two rotations for an insertion and three for a deletion (Cormen et al., 2009). Since none of this depends on random choices, the cost for a given input is fixed, so we expect the tree's latency distribution to have a tighter tail than the skip list's. However, this efficiency comes at the cost of increased implementation complexity, as the two fix-up routines, each with mirrored left and right cases, make up most of the code. To check them, we wrote an invariant checker that recomputes black heights and verifies ordering and parent links, and ran it after every deletion in an exhaustive test of all insertion and deletion orders over six keys.

1.3 Skip List

A skip list is a sorted linked list where, instead of each node holding a value and only a pointer to the next node, it holds a certain number of pointers depending on its 'height', and each pointer points to the next node in sorted order whose height reaches the same 'level' (Pugh, 1990) (Open Data Structures, n.d.). The height is a randomly generated integer, lower bounded at 1 and usually upper bounded, at 32 for example. The height is geometrically distributed, with p = ½. This is analogous to flipping a coin: if it is heads, we increase the level and flip again, and if it's ever tails, we terminate the process. Constant-time methods that give the same distribution can be used, such as generating a random 32-bit integer and counting its trailing unset bits, plus one.

A node of height k has k levels, zero-indexed from 0 to k − 1. Each level has a pointer to the first node with a larger value that also reaches that level. If there is no such node, it points to nullptr. This is what enables searching in expected logarithmic time. The process is analogous to binary search, as each level narrows the range that can contain x. To find the node that contains a value x, assign current to the head node, which is of the maximum height. Start at the highest level reached by any node other than the head, call it k. Current's pointer at level k points to the next node that reaches level k. If its value is less than or equal to x, assign current to this node; if it is greater than x, or the pointer is nullptr, go down a level. Repeat until the level is negative. x is present if current's value equals x.

This is expected O(log n) because there are expected O(log n) levels and about two moves per level: one downward move and an expected one rightward move. There are O(log n) expected levels because, of the nodes with height at least k, about half are expected to also have height at least k + 1. This is because all nodes with height at least k + 1 had to win the coin flip after winning k − 1 coin flips, so it is expected for half the nodes with height at least k

to win the coin flip, and half to fail. Halving n nodes at each level leaves about one node after $\log_2 n$ levels, giving O(log n) levels. The expected 2 moves per level can be found by reverse path analysis, starting where the search ended, at level 0. If the node's height is greater than level + 1, its tower reaches the level above, so the search must have come down at this node and the reverse path goes up. If the height equals level + 1, the search must have arrived from the left, so the reverse path goes left. Each node reaches the next level with probability ½, independently of the others, so the number of rightward moves before each downward move is geometric, giving an expected value of one right move per level (Jeff Erickson, 2020).

2. Theoretical comparison and predictions

2.1 Theoretical comparison

On paper both structures run every operation in O(log n) time, so the difference between them lies in guarantees and constant factors.

	Red-black tree	**Skip list (p = ½)**
Search	O(log n) worst case	O(log n) expected; O(n) worst case
Insert	O(log n) worst case	O(log n) expected
Erase	O(log n) worst case	O(log n) expected
Restructuring per update	≤ 2 rotations (insert), ≤ 3 (erase); recolouring O(log n) worst case, O(1) amortised	2 pointer changes expected
Space	O(n)	O(n) expected

Guarantees. The tree's bounds are guarantees: its height never exceeds $2 \cdot \log_2(n + 1)$. The skip list's bounds are expectations over its own coin flips, and they hold for every input order. Its worst case is O(n), but it is very unlikely: the probability that any node reaches more than $\log_2 n + c$ levels is at most 2^{-c}.

Constant factors. A skip list search moves right about once per level before dropping down, over about $\log_2 n$ levels. It therefore takes about $(1/p) \cdot \log_{1/} n = 2 \cdot \log_2 n$ steps in expectation. A red-black tree built from random keys has paths close to $\log_2 n$, well inside its $2 \cdot \log_2 n$ bound. Two effects narrow the gap. With only < available, the tree makes up to two comparisons per node, so about $1.5 \cdot \log_2 n$ comparisons per search. After the skip list drops a level, the next node it compares is often the one it has just rejected, so it reads only about

((1 − p²)/p)·log₁/□n = 1.5·log₂n distinct nodes. On updates, both structures make O(1) pointer changes on average, so the number of writes does not separate them.

Memory. A tree node holds a key, a value, three links (left, right and parent) and a colour: 48 bytes with integer keys and values. A skip list node of height h holds a key, a value, its height and h forward pointers: 32 + 8h bytes. Heights average 2, so the average is also 48 bytes, but the nodes come in many sizes.

Input sensitivity. The tree's shape depends on the order in which keys are inserted. Balancing bounds the damage, but sorted input causes the most rotations. The skip list's shape depends only on its random seed, never on the input. Both should be sensitive to the cost of a key comparison, which is higher for string keys, and to cache behaviour once the structure no longer fits in cache.

2.2 Predictions

Before running our benchmarks and comparisons, we made the following 5 predictions based on our research and understanding of the two algorithms.

H1. Average performance: Red black tree will perform worse on average than skip list due to higher implementation overhead of recolouring and rebalancing the tree after insert/erase. However, since both are built upon linked lists, have the same average and worst-case time complexity, the gap between the two will stay similar. We expected the tree's rebalancing to be its main extra cost. Because both structures have logarithmic operations, we expected the gap to stay roughly constant as n grows.

H2. Tail latency: Since skip lists' performance requires a good distribution of levels, which is determined by randomness. Predicting that skip list performs better at CI 50, 99, but not at 99.9. We expected the skip list to win on typical operations. We also expected its rare unlucky level distributions to show up only in the extreme tail, where the tree's guaranteed height would win.

H3. Input order: Input order matters more for the red-black tree, as suboptimal order creates more rotations, which may have a cascading effect up the tree for large heights. The tree's shape depends on insertion order and the skip list's does not. We therefore expected sorted or reverse input to cost the tree extra rotations, and to matter less for the skip list.

H4. Seed versus input: Seed has a larger effect on the skip list's performance rather than input order, as suboptimal randomness can cause more memory allocations, as well as poor height distribution, causing the time complexity to be closer to O(n) rather than O(logn). Input order has a miniscule effect given well distributed heights. We expected the skip list's own random seed to explain more of its variation than the input did.

H5. Writes versus reads: The gap for insertions and erase is larger than for exists. This is because most of the overhead in the red-black tree is due to rotations when inserting or erasing keys. Following H1, we expected the tree to fall further behind on inserts and erases, where it rebalances, than on lookups, where it does not.

3. Experimental methodology

3.1 What we measured

We measured each structure in three ways. First, wall-clock time for each phase of a workload (insert, lookup, erase), reported as nanoseconds per operation. Second, the latency of every individual operation, giving its full distribution and not only the mean. Third, structural costs counted inside the code: key comparisons, nodes visited, rotations and recolourings for the tree, pointer splices for the skip list, heap allocations, and the memory each structure holds after its build. A further experiment counted the distinct nodes, cache lines and memory pages each operation reads.

3.2 Inputs

Every workload is generated from a seed as a complete list of operations before timing starts. Keys are distinct 64-bit integers produced by a bijective scrambler (splitmix64). Each of the seven workload families targets a feature we expected the two structures to respond to differently.

Workload	Operations	What it probes
Uniform	Insert n random keys; n lookups (half hits, half misses); erase all in a fresh random order	The baseline
Sorted, reverse	Insert and erase 0, 2, …, 2n−2 in ascending or descending order; misses fall in the odd-numbered gaps	Maximum rebalancing for the tree; the skip list's shape is independent of order
Sliding window	n rounds of inserting a new key and erasing the oldest	Churn at constant size
Zipf	n lookups skewed by Zipf(s ≈ 1); the oldest keys are the most popular	Skewed access
Zipf shuffled	As zipf, with popularity independent of insertion order	Skew without an age bias

Steady	On n keys, look up n/4 fresh keys, insert them, look them up again, erase them	The cost of a write beyond its search, since each write retraces its lookup's path

We used three key types: 64-bit integers, 11-character strings that fit inside std::string's small-string buffer, and 64-character strings that are always heap-allocated. The strings preserve the integers' order and differ within their first 11 characters, so comparisons cost about the same and only allocation differs. Sizes run from 10^3 to 10^6 in steps of about ×3, plus $3 \cdot 10^6$ and 10^7 for the uniform workload.

3.3 Setup

Every experiment ran on one machine: an AMD Ryzen 5 5600G (6 cores, 32 KB of L1 data cache and 512 KB of L2 per core, 16 MB of shared L3) with 16 GB of dual-channel DDR4-3200, running Windows 11 Pro. The code was compiled with GCC 16.2 (MSYS2 MinGW) as C++20 with -O3 -DNDEBUG, built with CMake and Ninja. Each run used High priority and was pinned to one core to reduce scheduler noise and core migration.

Each phase is timed as a whole with std::chrono::steady_clock and divided by its number of operations. A separate pass times every operation with the CPU timestamp counter (rdtsc, fenced with lfence), and subtracts the median cost of an empty timed region, about 20 ns, from each sample. The structural counters are compiled into a separate instrumented binary, so the timing binary contains no counting code and instrumentation cannot distort the times.

3.4 Repetitions and variance

For n up to 10^6, each configuration uses 2 algorithm seeds and 3 timed repetitions after 1 discarded warm-up, with 3 input seeds for the uniform workload and 2 for the other six. The largest sizes use one seed pair and 2 repetitions. The red-black tree ignores the algorithm seed, so for it these act as extra repetitions. Structural counts are fully determined by the seed pair, so the counting runs use one repetition of each of 2 input seeds × 2 algorithm seeds.

We report the median time over all seeds and repetitions, with a 5th–95th percentile band. Per-operation latency is reported as percentiles from p50 to p99.99 and the maximum. A separate experiment runs $n = 10^3$ to $3 \cdot 10^5$ with 5 input seeds × 20 algorithm seeds. It compares the spread across algorithm seeds (p95 ÷ p5) with the spread across input seeds, separating randomness in the skip list from randomness in the input. A follow-up with 200 algorithm seeds gives bootstrap confidence intervals on that spread.

3.5 Fairness

Each workload is generated once and converted to the chosen key type before any timing, and every structure then replays exactly the same operations in the same order. Both structures implement one shared interface, store a 64-bit value with each key, take keys by

const reference and follow the same semantics: inserting an existing key overwrites its value.

Both are textbook versions. The tree follows CLRS with a sentinel leaf. The skip list follows Pugh, with p = ½ and at most 32 levels, and stores each node's tower in the same allocation as the node. Key generation, conversion and CSV output all happen outside the timed regions, and each lookup's result is added to a checksum so the compiler cannot discard it. Before benchmarking, both structures pass tests that compare them step by step against std::map over 50,000 random operations and a replay of every workload, with their invariants checked throughout.

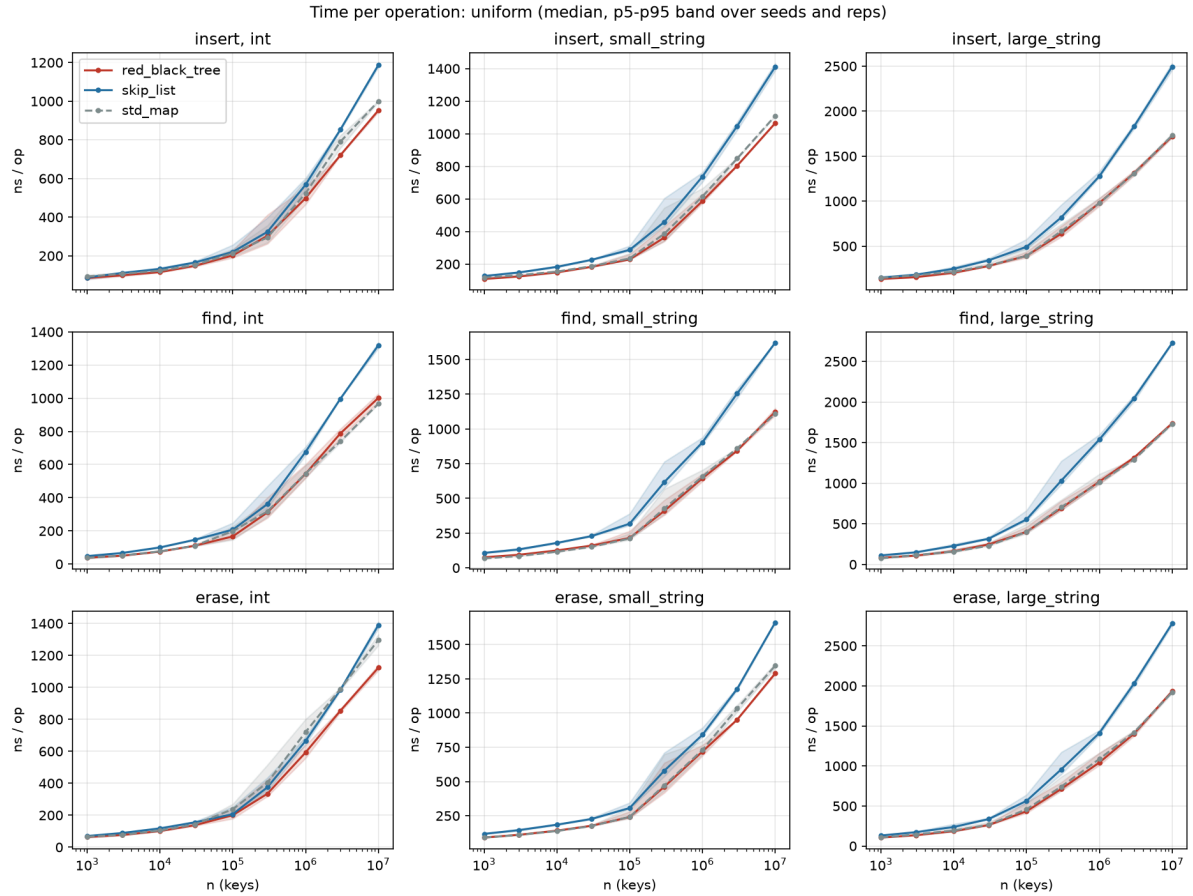
4. Results and analysis.

The red-black tree is faster than the skip list in almost every regime we tested, by up to 1.6× on uniform input, and four of our five predictions were contradicted; the fifth held, but for a different reason than we gave. Figure files are in `results/plots/`.

4.1 Overall speed

On uniform random input the red-black tree is faster than the skip list in every phase, at every size and with every key type. With integer keys at $n = 10^6$, a tree insert takes 498 ns against the skip list's 568 ns, a lookup 544 against 676 ns, and an erase 592 against 665 ns: the skip list is 1.14×, 1.24× and 1.12× slower. This contradicts the first half of H1, which

predicted that the tree's rebalancing would make it the slower structure.

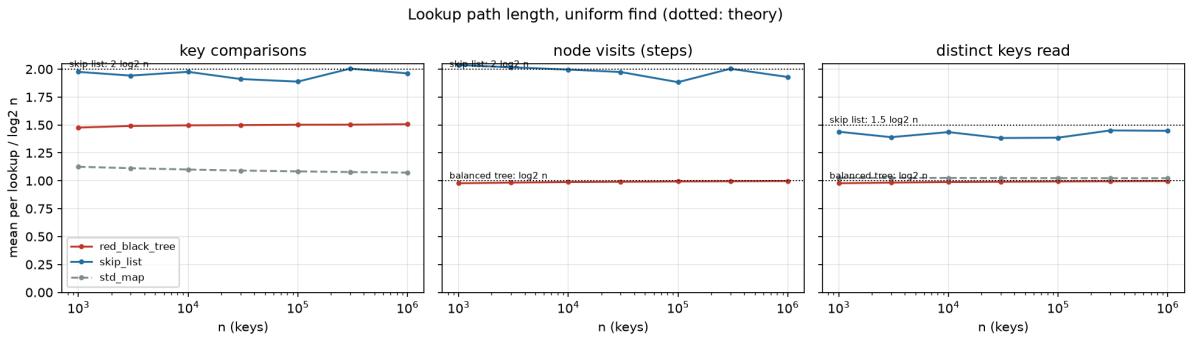


The second half of H1, that the gap would stay similar as n grows, holds for integer keys. The lookup ratio stays between 1.16 and 1.34 from 10^3 to 10^7 keys, while absolute times grow about 26-fold. It also roughly holds for 11-character strings (about 1.4–1.5×). It fails for 64-character strings, where the gap widens from 1.35× at 10^3 to 1.57× at 10^7.

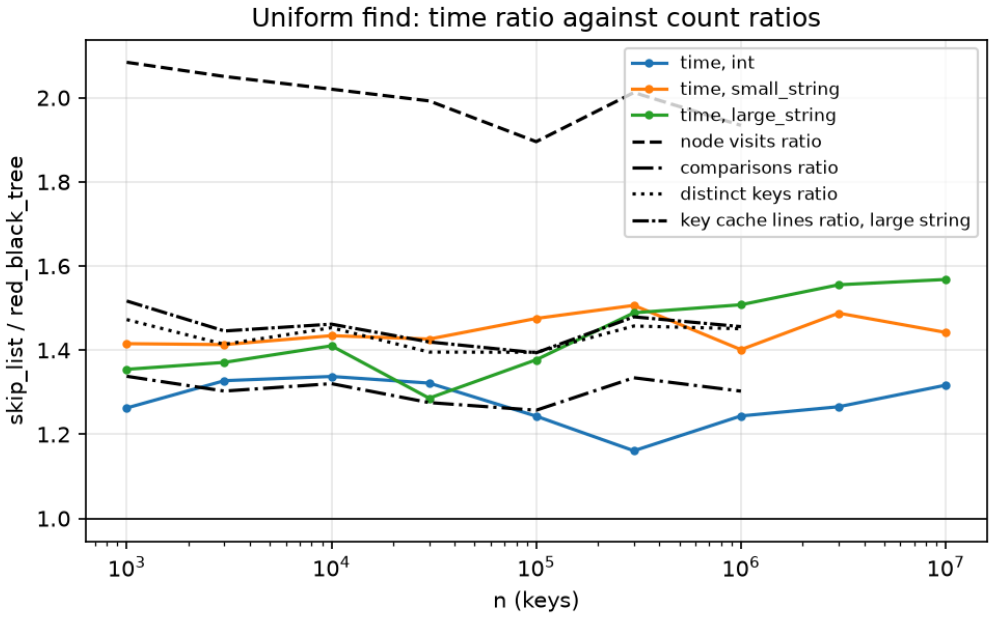
The gap is largest for lookups and smallest for writes, the reverse of H5. Sections 4.2 and 4.3 explain both results.

4.2 Why the tree is faster

H1 assumed that the tree's extra work lay in rebalancing, and that a search costs about the same in both structures. The structural counters show the opposite. A uniform lookup in the tree visits 0.98–1.00·$\log_2 n$ nodes at every size from 10^3 to 10^6, close to the depth of a perfectly balanced tree. The skip list visits 1.88–2.04·$\log_2 n$. As section 2.1 showed, a skip list search with $p = ½$ moves right about once per level before dropping down, so twice as many steps is built into the design rather than caused by unlucky seeds. This factor of two is a standard result for skip lists (Pugh, 1990). When forming H1, we overestimated the complexity of rebalancing.



The time gap is much smaller than this 2× step ratio because not every step costs the same. The tree makes up to two comparisons per node, giving $1.5 \cdot \log_2 n$ comparisons against the skip list's $1.95 \cdot \log_2 n$, a ratio of about 1.3. When the skip list drops a level, the next key it compares is often the one it has just rejected, which is already in cache. It therefore reads only $1.38–1.45 \cdot \log_2 n$ distinct nodes. For integer keys the time ratio (1.16–1.34, mostly near 1.3) tracks the comparison ratio.

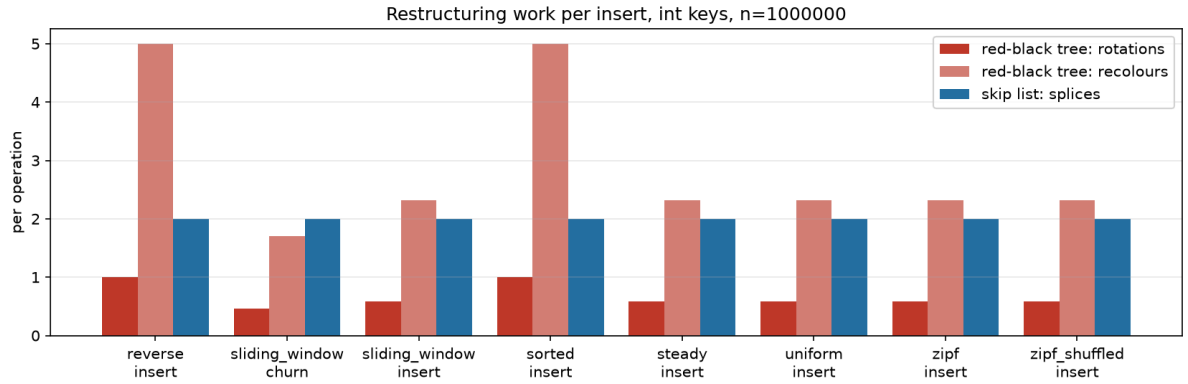


For 64-character keys, every distinct key read also means loading a separate string buffer. Here the skip list touches 2.15 times as many memory pages as the tree, because its nodes usually sit on a different page from their string buffers, while the tree's usually share one. This extra page traffic is the likely reason the gap is larger for 64-character keys and grows as the structure outgrows the cache.

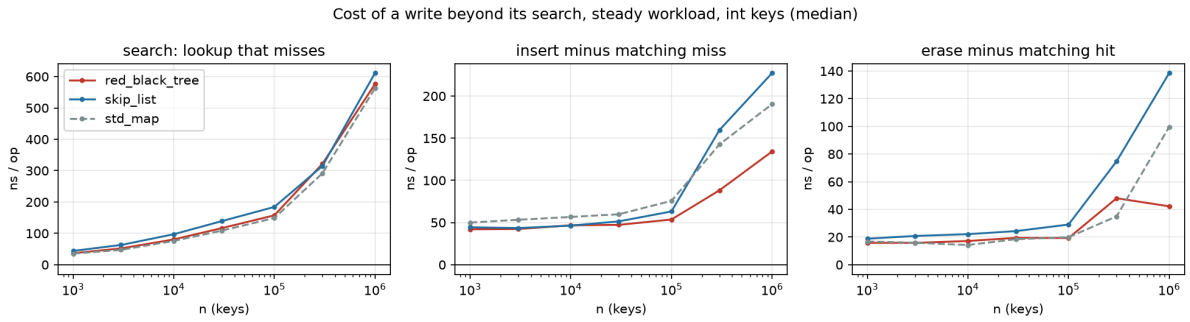
4.3 Writes and rebalancing

H5 predicted that the gap would be larger for inserts and erases, because the tree's rotations would dominate its write cost. The data shows the reverse: the skip list is 1.24× slower on uniform lookups but only 1.14× on inserts and 1.12× on erases. The tree's rebalancing turns out to be cheap. A uniform insert causes 0.58 rotations and 2.3 recolourings on average.

Sorted and reverse inserts cause the most, 1.0 rotation and 5.0 recolourings each, which confirms H3's premise that ordered input increases rotations. All of this work touches nodes that the search has just visited, so they are already in cache. The skip list splices 2.0 pointers per insert on every workload.



The steady workload isolates the cost of a write beyond its search. Up to 10^5 keys, the extra cost of an insert is nearly the same for both structures (54 ns for the tree, 63 ns for the skip list at 10^5). Beyond that the skip list's writes become more expensive: at 10^6, its insert costs 227 ns beyond the search against the tree's 134, and its erase 139 against 42. So the tree's writes are never its weak point, while the skip list's become costly from about $3 \cdot 10^5$ keys. Section 5.3 discusses why.



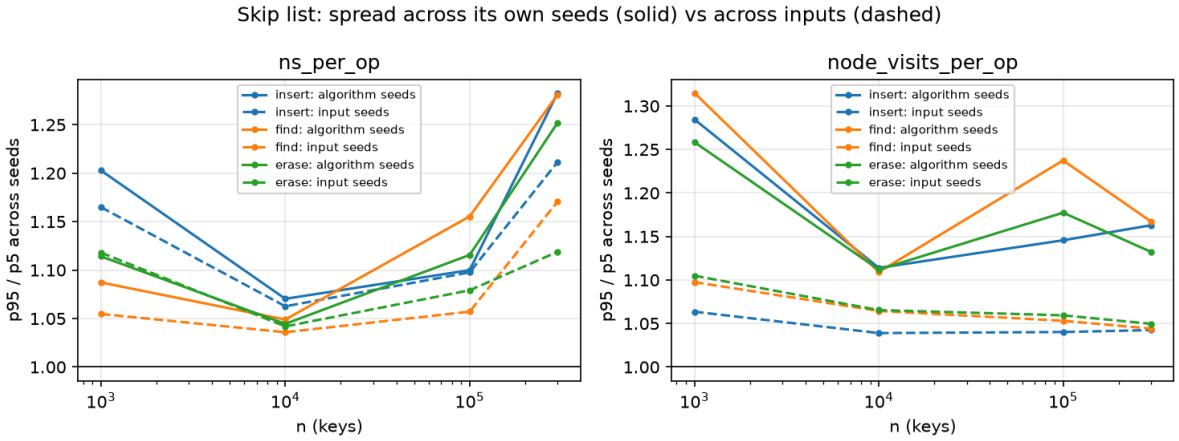
4.4 Input order and access skew

H3 predicted that input order would matter more for the tree. Order does matter, but for both structures, and not through rotations. Despite its extra rebalancing, the tree inserts sorted keys six times faster than random ones (80 against 498 ns). The skip list speeds up just as much (84 against 568 ns), and on reverse input it beats the tree (48 against 71 ns). Consecutive inserts follow the same path, which stays in cache, so the rotation count barely shows in the time. For lookups, building the structure in order leaves the tree's speed roughly unchanged (542–587 against 544 ns) but makes the skip list 17–20% faster, enough to match the tree. Access skew also favours the tree under zipf (2.44×), but only because of how that workload was built (section 5.2).

4.5 Tails and randomness

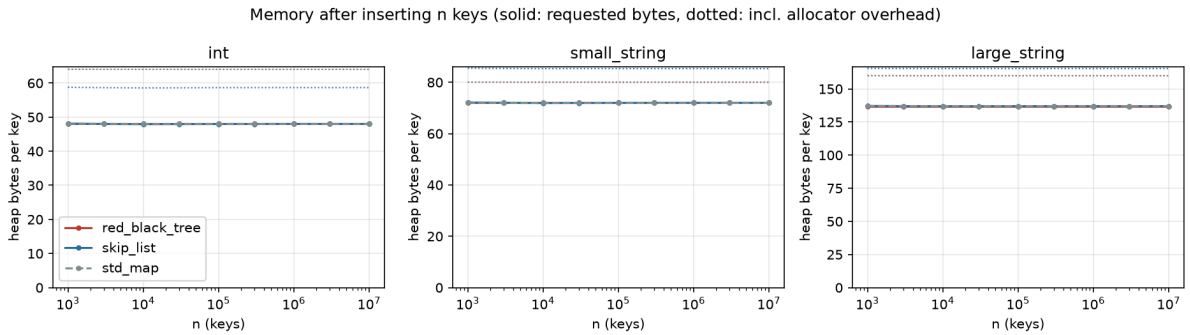
H2 predicted that the skip list would be faster at the median and the 99th percentile, and lose only in the extreme tail. In fact the tree is faster throughout. For uniform lookups, the tree's median is 566 ns against the skip list's 691, and its 99th percentile 1,047 against 1,663 ns. At the 99.9th percentile the gap is widest, 1,267 against 2,249 ns. Only at the 99.99th percentile do the two converge (5,340 against 6,112 ns), where system noise starts to dominate. The comparison counts show why the skip list's tail is heavier. The tree's worst lookup made 46 comparisons, held down by its guaranteed height. The skip list's worst made 74, about $3.7 \cdot \log_2 n$. That is a long tail, but nowhere near the O(n) degradation that H4 feared.

H4 predicted that the skip list's random seed would affect it more than input order. This holds: at $3 \cdot 10^5$, the spread (95th ÷ 5th percentile) of node visits per lookup is 1.17 across algorithm seeds and 1.04 across input seeds. H4's proposed mechanism does not hold, though. The skip list allocates exactly one node per key whatever the seed, so the seed changes only the heights. With 200 seeds, the spread shrinks steadily as n grows, from 1.34 at 10^3 to 1.16 at $3 \cdot 10^5$. A search is a sum of many independent random steps, and their total concentrates around its mean.



4.6 Memory

At 10^6 integer keys, both structures request 48 bytes per entry on average. The tree always stores three links and a colour, while the skip list's towers average two forward pointers. Including the heap's own bookkeeping, the skip list uses 58.7 bytes per entry against the tree's 64. With string keys the order reverses: 85.3 against 80 bytes for short strings, and 165.3 against 160 for long ones.

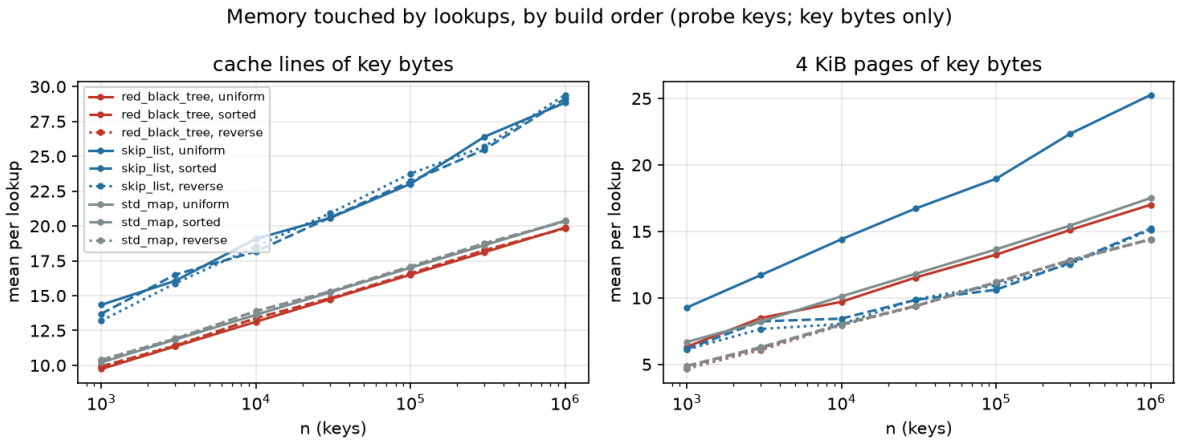


5. What surprised you

5.1 Node size shapes memory layout, and layout shapes speed

Our most surprising result has nothing to do with either algorithm's textbook analysis. After a sorted or reverse build of 10^6 integer keys, skip list lookups take 564 and 543 ns, against 676 ns after a random build. That is 17–20% faster, enough to match the tree. Yet the skip list reads the same number of nodes in all three cases: 29.2, 29.4 and 28.9 distinct keys per lookup. The tree's lookup time barely changes with build order.

Counting the memory each lookup touches located the difference. Every key read sits on its own cache line whatever the build order, so cache lines cannot explain it. Memory pages can. After a sorted build, a skip list lookup touches 15.2 pages instead of 25.3, 40% fewer, while the tree's falls by only 15%. The difference comes from node size. Every tree node is 48 bytes, and 99.8% of consecutive tree allocations land within one page of each other. A skip list node is 32 + 8h bytes for a tower of height h, so the nodes come in many sizes. Only 40% of consecutive skip list allocations land near each other, close to the roughly one-in-three chance that two random towers have the same height. The heap appears to keep each block size in its own area. After a sorted build, nodes of the same height sit in key order within their area, so neighbouring keys share pages. After a random build, nothing does.



The same cause accounts for the larger gap for 64-character keys in section 4.2. For the tree, 64% of the keys read share a page with their string buffer; for the skip list, only 6% do, so each skip list key read costs about two pages instead of one. The skip list's random tower heights, the source of its simplicity, therefore also scatter it through memory. The placement is measured, but two links in this chain are inferred: that the Windows heap groups blocks by size, and that the speed difference comes from page-translation (TLB) misses.

5.2 Our own workloads misled us twice

Twice, a result that looked like a property of the algorithms turned out to be a property of our benchmark.

In the zipf workload, the tree's lead was its largest anywhere: 2.44× at 10^6. The workload gave popularity rank r to the r-th key inserted, so the hottest keys were the oldest. Early keys tend to stay near the root of a red-black tree, so a hot lookup visited 12.8 nodes on average instead of 19.9. A skip list node's height does not depend on when it was inserted, so the skip list could not benefit. When we assigned popularity by an independent shuffle (zipf shuffled), the tree's node visits returned to 19.4 and its lead fell to 1.36×, close to its uniform lead.

In our first run, the tree's lookups after a sorted build appeared twice as fast as after a random build (263 against 544 ns). The sorted workload used keys 0 to n−1 with random 64-bit misses, so every miss was larger than every key and followed the same path down the right edge of the tree, which stayed in cache. Once the keys became 0, 2, …, 2n−2 with misses on the odd numbers between them, the difference disappeared (587 against 544 ns). Each flaw encoded an assumption that happened to favour the tree.

5.3 A cost we still cannot explain

The steady workload measures what a write costs beyond its search. Up to 10^5 keys, both structures pay about the same. From $3 \cdot 10^5$ the skip list's writes become far more expensive: at 10^6, an insert costs 227 ns beyond its search against the tree's 134, and an erase 139 against 42.

We tested and ruled out three explanations. First, extra search work: each write does the same search as its matching lookup, with comparisons, distinct keys and pages all within 1%. Second, allocation: in isolation, allocating either node shape costs about 42 ns. Third, allocation after a cache-missing search: with 4 or 16 cache-evicting reads before each allocation, neither node shape is consistently dearer, and the differences (at most about 15 ns, against noise of ±20 ns) are far smaller than the 93 ns insert gap (`alloc_evict.png`). An earlier explanation, suggested by an AI assistant, was that a write touches predecessors that are not on the cached search path. It contradicts the definition of a predecessor, the node where the search dropped down a level, which the search has just read.

The cause remains open. Counting cache and TLB misses per operation with hardware counters, or giving both structures the same pool allocator, would settle it (section 6).

6. What we'd do next

Settle the write-cost question. Section 5.3 left the skip list's expensive writes beyond $3 \cdot 10^5$ keys unexplained, after ruling out extra search work and allocation. Two experiments would settle it. The first is to count cache and TLB misses per operation with the processor's hardware counters (AMD uProf on our Ryzen 5 5600G), which would show directly whether the skip list's writes miss more often, and where. The second is to give both structures the same pool allocator, so that any remaining gap must come from the algorithms themselves.

Separate the algorithm from the allocator. Section 5.1 traced several results to how the Windows heap places nodes of different sizes. We cannot yet tell whether that is a property of skip lists or of this heap. Rerunning the experiments with a different allocator, and on Linux with glibc's allocator, would show whether the skip list's scattered layout, and its speed-up after a sorted build, appear everywhere or only on Windows.

Vary the skip list's design. Our skip list fixes $p = ½$. Making p a parameter would test the trade-off the theory predicts: with $p = ¼$ a lookup should read about $1.875 \cdot \log_2 n$ distinct nodes instead of $1.5 \cdot \log_2 n$, while each node averages 1.33 forward pointers instead of 2. Since our measured counts already sit about 6% below the formula, this would also test the formula. A second change would have the search remember the node it rejected just before dropping a level. That would save about $0.5 \cdot \log_2 n$ repeated comparisons per lookup, which matters most for string keys, where each comparison may cost a memory access.

Check the results on a second compiler. All our results come from GCC 16.2. Our MSVC comparison harness (dict_compare) could rerun the head-to-head experiment on the same machine, to test whether the speed ratios hold across compilers and standard libraries.

Test concurrency. Concurrency is the main reason skip lists are used in practice, for example in Java's ConcurrentSkipListMap and the in-memory tables of LevelDB and RocksDB, and our single-threaded results say nothing about it. Both structures make O(1) pointer changes per update on average, but their shapes differ. A skip list's changes are independent single-pointer swaps: a new node can be linked from the bottom up with compare-and-swap, and it is in the set as soon as level 0 is linked. A tree rotation changes several pointers that must change together, and where the fix-up will happen is not known in advance. A multi-threaded benchmark could show whether the skip list's single-threaded deficit turns into an advantage.

Try cache-conscious designs. Section 5.1 suggests that memory layout matters more than step count. Unrolled skip lists, which store several keys per node, or a B-tree such as Abseil's btree_map would test that directly.

Cover more operations and inputs. We did not benchmark range queries, in-order iteration or rank queries (supported by an indexable skip list), all of which ordered dictionaries exist to provide. We also do not know whether the skip list's advantage after a sorted build holds for nearly-sorted or append-heavy input.

7. Individual contributions

Aaron:

I created the initial repo for our team to use. I wrote the report based on Laith’s comparison and testing implementation. I also assisted in creating the initial hypotheses we tested. I did implement comparison functionality, but we ended up using Laith’s instead.

Laith:

I wrote the overview in the report, and created the hypotheses, tests, theories, and further tests. Hence, I largely did the experimental part directly, whilst Aaron did the writeup in the report.

8. AI use

Laith:

I used Claude initially to guide selecting the topic, understand what a skip list was, and understand general implementation decisions such as how to manage memory correctly, and whether I should include assertions in code. I used Claude Code to assist me with implementation after coding most of the skip list myself. Then I used it heavily to create cmake files, unit tests, and the benchmark tests, as well as expand on my hypotheses, and give several theories of why some of my hypotheses failed, and how we could test those theories.

I used Claude Code at the end to double check my scripts for the presentation, to generate the visuals for the video, and then combine the videos and audio into one file.

Examples of Being Wrong:

Claude Code replaced a check for a duplicate key being inserted in skip list with a comparison that just used the less than operator. I didn't understand how that would check for a duplicate value, so I asked it to explain how that would work as a check, which it then said that search() returned the first key which is not less than element (at this point in time I was only storing keys which I called element, instead of key value pairs), which was wrong as search() return the strict predecessor of element. It then explained that it meant that the line after search moved the node forward, but still, this wouldn't function as a useful duplicate check, as if we use the less than operator, and there's a duplicate, it will return false that the duplicate is present. I explained to Claude how it wouldn't function as a duplicate check, and then it undid those changes.

Reflection on AI in Group Work Specifically:

I found parts of it annoying, as it felt like the AI overstepped in its implementation before I even understood my data structure. There was already a generic implementation alongside benchmarking before I understood anything. I feel like there has to be a point of understanding what is going on before using AI to assist, otherwise it feels like managing something you don't understand.

AI was extremely useful however for explaining my teammate's implementation and how there could be parts that could skew the experiment without being strictly due to the data structure, but just inconsistent implementation. For example, in my original skip list class, I was using vectors to store the next nodes, which meant I was doing two heap allocations when creating a node, whilst Aaron's only used one. So the analysis wouldn't be a useful comparison between data structures, but just how a well implemented algorithm would exceed a poorly implemented one.

Aaron:

I used Claude mainly for research and code generation. I used Claude to research the red black tree algorithm, both the theory and implementation. For the theory I used it to guide me on what resources I could use. For the implementation I used it to assist in debugging and polishing the core algorithm code. I then used it heavily for creating tests and for creating the structure for comparisons between the two algorithms (though this ended up not being used as we went with Laith's comparison implementation. I also used it in a limited capacity in helping write the report. Mainly, I used it to assist with editing and polishing the report.

Examples of Being Wrong:

When I asked Claude to look over this report and help with editing/publishing, it gave recommendations that were strange and not overly helpful, mainly due to it misunderstanding the intent behind different parts of the report. For example, it wanted to add in detail to our predictions that we had missed, despite it being explained that this was not correct for us to add information after the fact. In terms of implementing the algorithm, which I used Claude to help with, it was good overall and made minimal mistakes. Once again, the issue I had was with alignment. Instead of focusing on helping me implement the algorithm I had selected, it continuously attempted to make changes that were outside the scope of this assignment.

Reflection on AI in Group Work Specifically:

There are a number of ways in which using AI in a group setting was difficult. When using it for generating and explaining code, I found I had to continuously realign what I was trying to get the AI to do. It had a tendency to go into details and files that I had explicitly told it to leave untouched (as they were my teammates responsibility). More broadly speaking, I think the difficulty of using AI in group work is that it adds another layer of communication. Laith and I had to come to an understanding of what we were doing, and I then had to communicate with my AI model of what the understanding was. Each step gives more room for misunderstanding and unnecessary complexity that adds to the work needed rather than removes it.

However, I would say overall it is much more of a help than a hindrance. The main way I found AI helpful was in research and analysis. I was able to use it to find resources that would help me understand complicated topics, using its recommendations as a starting point for my own research. It was also a great help in collecting and correlating the actual data from our comparisons to be readable and usable for this report.

Reference List:

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to algorithms (3rd ed.). MIT Press.

Jeff Erickson. (2020). Algorithms lecture 3: Treaps and skip lists [Sp’20]. https://courses.grainger.illinois.edu/cs473/fa2024/notes/03-treaps.pdf

Open Data Structures. (n.d.). 4 Skiplists. In Open data structures. Retrieved November 4, 2026, from https://opendatastructures.org/newhtml/ods/latex/skiplists.html

Pugh, W. (1990). Skip lists: A probabilistic alternative to balanced trees. Communications of the ACM, 33(6), 668–676. https://doi.org/10.1145/78973.78977